

CS 6160: Introduction to Cryptology

Homework Assignment 1

Shibam Ghosh
IIT Hyderabad

July 28, 2026

Please provide proper instructions on how to run the program. The submission deadline is 6th August, 2026.

1 Homework 1 (30 Points)

In classical cryptanalysis, breaking substitution ciphers (such as the monoalphabetic substitution cipher) relies on understanding the underlying statistical distribution of the target language. In standard English, for example, the letter 'E' accounts for approximately 12.7% of all alphabetic characters, followed by 'T' ($\approx 9.1\%$) and 'A' ($\approx 8.2\%$).

Your task is to write a **C or C++ program** that reads an unparsed, raw text file (e.g., raw text extracted from Wikipedia or Project Gutenberg) and computes the empirical relative frequency percentage of each letter ($A-Z$).

1.1 Requirements

1. **File Input:** Your program must accept a text file path via command-line arguments (e.g., `./freq_analyzer input.txt`).
2. **Case Insensitivity:** Handle both uppercase ($A-Z$) and lowercase ($a-z$) characters, normalizing them into the same frequency bin (i.e., 'a' and 'A' must both increment the count for letter A).
3. **Non-Alphabetic Filtering:** Ignore punctuation, numbers, whitespaces, newlines, and special characters. They must **not** contribute to the total count of alphabetic characters.
4. **Relative Frequency Calculation:** For each letter $c \in \{A, B, \dots, Z\}$, compute its relative frequency percentage $f(c)$ using the formula:

$$f(c) = \left(\frac{\text{Count}(c)}{\text{Total Alphabetic Characters}} \right) \times 100\%$$

5. **Output Formatting:** Print a clean, formatted table displaying each letter, its absolute count, and its calculated relative frequency percentage rounded to two decimal places.

1.2 Hints & ASCII Table Inspection

Before writing your program, inspect the standard ASCII character table in your Linux terminal by running:

```
$ man 7 ascii
```

Pay close attention to the decimal ranges for the English alphabet, this will help you with the steps 2 and 3:

- **Uppercase range ('A'–'Z'):** Decimal values **65** to **90**.
- **Lowercase range ('a'–'z'):** Decimal values **97** to **122**.

2 Homework 2 (70 Points)

This is the ciphertext of a Vigenère cipher:

```
DVNSJ QRVPC GWBVZ QGVVP VILQJ VTURP BNPZN TGQIG SIGVW MTCIM FQUIA CCKVG CBYKQ
ZNTNB UCIEN UHIVF IWPQC BNKCM IGGGV OEWEV PVGFX APQKM EATDR TBIQG IPRNX JECGQ
NPLMY EDURF IPRXX AVVDZ JKIPN UBQYG PVQGM XYCXV RFIPN VIPRI GMNVT AGVGM NUJZR
KCBUG QCVNS QAILI FPDBN UXVTN TJBQZ JHVIP RMCWJ NTLTG IPNVR WAPTK GGSBU GBIYN
TDRTN ZBQBK BPIIV PTLZC CCFEG QCVHW AFXNS GGMAY HCOLT KGUXV PNJLV PVUNV WMZCI
QPUPA GTDVB ONXUK AWFQE PLOTL VEXVR GCOVP TMEKC ONPSP VUIWE AIPRU RPBPN ZQGRQ
QGSBB UEMAF IPRFP GRZET BTXVT GKMEA RWEPT ZBHIP RNXJE CGGGJ TNVTH BEQDU PQCBN
KCMQD DWXUD VZCIP ROPBV EHBUG HKUQA IETTI QCQWH VCCZD TZFIT WZGIZ VEUQT WGMFC
CLRNT ONPIX EQDNF VWIGJ PLVPH XVTTT TGCME CIQBP HWSVW QAMTZ FQCMZ CCCFE GQCVS
MFEGQ OGSPB YHQZR AMBDH MEXPB VQCAP QJTQN TIQVD LRGEV UGDZR OHEUK AMNPD BUGGM
KRAIV PTLUQ LIOUI ZNEIQ QGPAB HIMAH DCAFE ZNEIQ PCAIC RAQPC IQBPH KRPIC EKTAY
CIMEV WMFEW WYCGZ RCAQF GSBUC IUNVW MZCIQ PULIF PDBZG GMYAP KBNAM PVXWA QUNBT
BCYCH JHVPT NPVCN ITNBT SMFEG QOKCO CCIBR TCANP SZRCH WAKCO NDDCG VWMJQ GTQ
```

Your task is to write a **Python** program that breaks the Vigenère cipher and finds the encryption key. The program must decrypt the ciphertext using the Vigenère cipher decryption algorithm (i.e., you have to implement a decryption function) and output the plaintext.

3 Hint: Tutorial on Breaking the Vigenère Cipher

This tutorial is to help you understand the classical attack against the Vigenère cipher. The classical attack against the Vigenère cipher is known as **Kasiski's Method**. The attack consists of two independent stages.

1. Estimate the length of the secret key.
2. Recover each letter of the secret key individually.

Once the secret key has been recovered, decrypting the ciphertext becomes straightforward. Suppose the secret key is MATH, whose length is $\ell = 4$. During encryption, the key is repeated. For example,

Plaintext	A	T	T	A	C	K	A	T	D	A	W	N
Key	M	A	T	H	M	A	T	H	M	A	T	H

Notice that

- positions 0, 4, 8, ... are encrypted using the letter M,
- positions 1, 5, 9, ... are encrypted using the letter A,
- positions 2, 6, 10, ... are encrypted using the letter T,
- positions 3, 7, 11, ... are encrypted using the letter H.

Therefore, if the key length is known, the ciphertext can be divided into ℓ independent groups. For $\ell = 4$, we obtain four slices or buckets

$$\begin{aligned} S_1 &: C_0, C_4, C_8, C_{12}, \dots \\ S_2 &: C_1, C_5, C_9, C_{13}, \dots \\ S_3 &: C_2, C_6, C_{10}, C_{14}, \dots \\ S_4 &: C_3, C_7, C_{11}, C_{15}, \dots \end{aligned}$$

Each group has been encrypted using exactly one shift. Hence,

Breaking one Vigenère cipher is equivalent to breaking ℓ independent shift ciphers.

3.1 Step 1: Estimating the Key Length

The first challenge is determining the unknown key length. To do this we use the *Index of Coincidence (IoC)*. Suppose a text contains N letters. Let n_i denote the number of occurrences of the i -th letter of the alphabet. The *Index of Coincidence* is defined as

$$I = \frac{26 \sum_{i=0}^{25} n_i(n_i - 1)}{N(N - 1)}.$$

The IoC measures the probability that two randomly selected letters from the text are identical. Typical values are

$$\begin{aligned} \text{Random text} &\approx 1, \\ \text{English text} &\approx 1.7. \end{aligned}$$

3.1.1 Estimating the Key Length Using IoC

Assume the candidate key length is ℓ . Split the ciphertext into ℓ slices. For example, if $\ell = 4$,

$$\begin{aligned} S_1 &: C_0, C_4, C_8, C_{12}, \dots \\ S_2 &: C_1, C_5, C_9, C_{13}, \dots \\ S_3 &: C_2, C_6, C_{10}, C_{14}, \dots \\ S_4 &: C_3, C_7, C_{11}, C_{15}, \dots \end{aligned}$$

If the guessed key length is correct, each slice has been encrypted using a single shift cipher and therefore resembles ordinary English. Consequently, its IoC will be close to that of English. If the guessed key length is incorrect, each slice mixes several shift ciphers together, producing text that is closer to random. Consequently, its IoC will be much smaller. Therefore, we compute the average IoC for every candidate key length and select the smallest candidate whose average IoC is close to that of English.

3.1.2 Algorithm for Estimating the Key Length

Input: Ciphertext C , maximum key length L .

Output: Estimated key length ℓ .

1. For every candidate key length $j = 1, 2, \dots, L$:
 - (a) Split the ciphertext into j slices.
 - (b) Compute the loC of every slice.
 - (c) Compute the average loC.
2. Return the smallest key length whose average loC is closest to English.

3.2 Step 2: Recovering Each Key Letter Using Frequency Analysis

Once the key length ℓ has been determined, the ciphertext is divided into ℓ independent slices. Since all letters in a slice are encrypted using the same shift, each slice is simply a shift cipher. The goal is therefore to determine the shift for each slice. Suppose S is one ciphertext slice. Since a Caesar cipher has only 26 possible shifts, we simply try every possible shift. For every candidate shift $k \in \{0, 1, \dots, 25\}$, we perform the following steps.

1. Decrypt the slice using shift k .
2. Compute the letter-frequency vector $\gamma^k = (\gamma_0^k, \gamma_1^k, \dots, \gamma_{25}^k)$, where γ_i^k denotes the relative frequency of the i -th letter in the decrypted slice.
3. Let $\delta = (\delta_0, \delta_1, \dots, \delta_{25})$ denote the standard frequency distribution of English text.
4. Compare the two frequency vectors using the *cosine similarity*

$$\cos(\theta) = \frac{\gamma^k \cdot \delta}{\|\gamma^k\| \|\delta\|}, \text{ where } \gamma^k \cdot \delta = \sum_{i=0}^{25} \gamma_i^k \delta_i.$$

The cosine similarity measures how closely the decrypted slice resembles ordinary English. If the value of cosine similarity is close to 1 (say more than 0.9), it indicates that the decrypted slice has a letter distribution closer to English and is therefore more likely to have been decrypted using the correct Caesar shift.